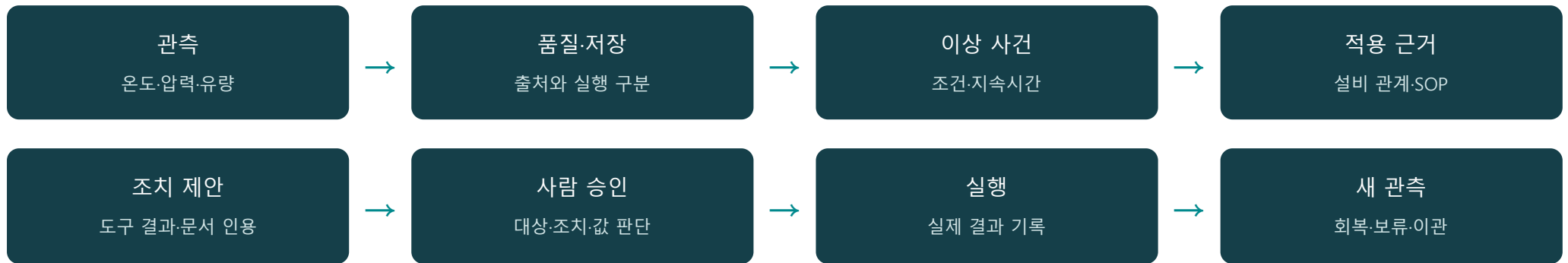


온도 한 줄에서 근거 있는 조치와 회복 확인까지

학생은 제공된 제조 AI 플랫폼을 직접 사용하고, 결과를 설명하며 자기 문제에 적용합니다.

교육 상황은 유압설비의 냉각 이상 대응입니다. 온도가 높다는 이유만으로 AI가 바로 조치하지 않습니다. 값의 품질, 설비와 문서의 관계, 사람의 승인, 조치 후 새 관측이 연결되어야 합니다.



위 두 줄은 이어지는 하나의 흐름입니다. 새 관측으로 다시 상태를 판단하는 연결을 페루프라고 부릅니다.

UCI 파일 재생

기록된 실제 센서값을 읽어 집계·품질·출처 추적을 배웁니다. 과거 파일을 다시 읽어서 이번 조치의 효과를 입증할 수는 없습니다.

교육용 합성 시뮬레이터

프로그램이 새 관측을 만듭니다. 구현된 조치와 이후 값의 변화를 연결해 회복 확인을 연습합니다. 실제 공장의 효과로 일반화하지 않습니다.

품질 OK는 제공 센서 품질 규칙을 통과했다는 뜻입니다. 설비 정상이나 조치 후 회복을 뜻하지 않습니다.

누가 만들고, 누가 판단하는가

학생

목적·입력·원하는 동작·확인 기준을 정합니다. 실행 결과와 근거를 읽어 수용하거나 수정을 요청합니다.

코딩 에이전트

개발 도구 안에서 설치·코드 작성·MCP 연결·검사를 수행합니다. 학생이 요구한 시스템을 만드는 역할입니다.

업무 에이전트

만들어진 플랫폼 안에서 사건의 근거를 조회하고 조치를 제안합니다. 코딩 에이전트와 다른 실행 주체입니다.

각 기술은 다음 판단에 필요한 정보를 연결합니다

구성	맡는 역할	학생이 확인할 것
Python · PostgreSQL	관측 처리와 실행·사건 기록 저장	같은 센서·실행·시간 구간을 다시 찾는가
온톨로지 · Neo4j	설비·센서·허용 조치·문서 관계 조회	이 설비에 적용할 수 있는 근거인가
SOP 검색 · RAG	관련 절차의 문서와 절을 찾아 제안에 연결	문서의 대상·버전·내용이 제안을 뒷받침하는가
데이터 패브릭 · 도구 연결	흩어진 데이터와 관계·접근 경로를 연결해 활용	필요한 정보의 위치와 조회 경로를 설명할 수 있는가
MCP · 업무 흐름 · 관제	도구 호출, 승인·실행·재측정과 결과 관찰	제안·승인·실행·회복 기록을 구분하는가

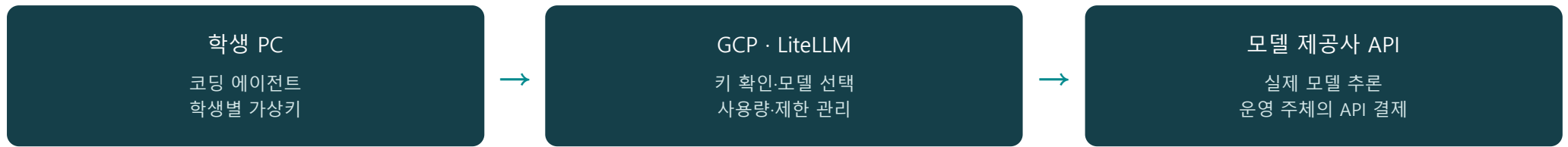
짧은 명세에서 개인 산출물로

예: “사이클100의 온도 그래프를 만들고, 선택한 값의 단위·원본 위치를 확인할 수 있게 해 줘.” 학생은 실제 출력과 원본을 대조합니다. 이렇게 구현 전에 요구와 확인 기준을 적는 방식이 SDD입니다.

공통 실습에서는 안내를 충분히 제공하고, 개인 PBL에서는 익숙한 스택으로 자신의 요구를 정의하고 결과를 만듭니다. 개인 PBL의 구체 시간 배분은 별도 확정 대상입니다.

학생의 시스템은 PC에서, 모델 요청은 LiteLLM을 통해

LiteLLM은 모델 API로 가는 요청을 중계하고 학생별 접근·사용량을 관리하는 관문입니다.



응답은 반대 방향으로 돌아옵니다. Cloud Run에 올리려는 것은 중계 프로그램이며, 모델 추론용 GPU 서버가 아닙니다.

학생 PC에 두는 것

교육용 플랫폼·관제, 분석·시뮬레이터, PostgreSQL·Neo4j, 개인 코드와 산출물. MCP로 연결하는 로컬 도구도 이 실행 환경에 속합니다.

중앙에 두는 것

LiteLLM, 가상키·사용량을 보관할 운영 DB, 제공사 키 보관, 모델 접근 설정과 모니터링. 중앙 DB는 학생의 설비 관측 DB와 별개입니다.

요청 한 번의 경로

① 학생이 코딩 에이전트에 요구합니다. ② 에이전트가 필요한 프롬프트·코드 문맥을 중계 주소로 보냅니다. ③ LiteLLM이 키와 허용 설정을 확인해 제공사로 전달합니다. ④ 응답을 받은 코딩 에이전트가 로컬 도구를 실행하고 학생이 결과를 확인합니다.

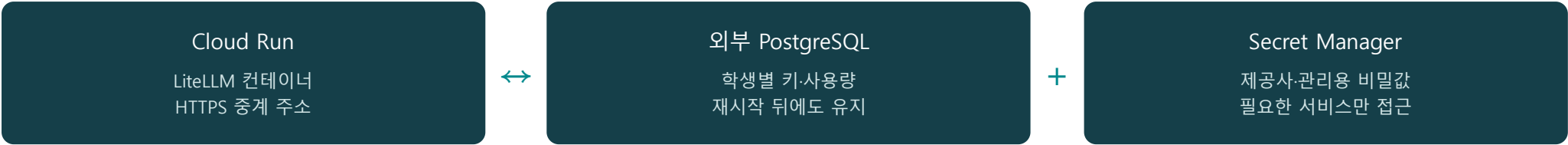
로컬 실행이어도 모델에 전달하는 프롬프트·선택된 코드·도구 결과는 외부 요청에 포함될 수 있습니다. “데이터가 전혀 PC 밖으로 나가지 않는다”는 구조가 아닙니다. 전달 범위와 로그 보관 정책을 정해야 합니다.

합의한 방향 학생별 로컬 + 중앙 LiteLLM, 코딩 기본 모델 GPT-5.6 Luna. **검증 필요** 실제 API 모델 ID·코딩 도구 호환·비용. 플랫폼 내부 업무 에이전트의 중계 연결도 코딩 도구와 별도로 시험해야 합니다.

기술 근거: [LiteLLM Virtual Keys](#) — 가상키·모델 접근·사용량 관리와 PostgreSQL 요구사항.

공유 주소를 만들기 전에 확인할 것

배포 후보 Cloud Run에서 LiteLLM을 실행하는 기존 운영 제안을 설명합니다. 현재 중앙 중계의 배포·동시 사용·비용 검증 완료 증거는 없습니다.



준비 단계	협업자가 확인할 결과
1. 운영 범위 결정	GCP 프로젝트·리전·운영 담당자·결제 주체·DB 방식·접근 정책. 학생용 중계와 관리자 기능의 접근 범위를 구분.
2. 배포 설정 준비	고정한 이미지 버전, 모델 매핑, DB 연결, 비밀번호 참조, 요청 제한시간·동시성·인스턴스 설정. 예전 가정값을 확정 사양으로 복사하지 않음.
3. 학생 키 연결	학생별 가상키, 중계 주소, 허용 모델. 원본 제공사 키와 관리용 마스터키는 학생에게 배포하지 않음.
4. 실제 사용 확인	일반 응답·스트리밍·도구 호출·긴 요청, 만료키·한도 도달·429·중계 장애 처리. 재시작 뒤 키와 기록 보존, 여러 인스턴스의 제한·집계 일관성.
5. 파일럿과 운영	학생 ID·회차·요청 ID·모델·입출력 토큰·지연·실패·비용을 대조. 사용자 실습 완주·비용 실측 후 사양과 예산 조정.

비용 항목: 모델 API + 중계 실행 + 운영 DB + 비밀·로그·네트워크 등. LiteLLM 사용량과 제공사 청구를 대조합니다. 설정한 예산이 정확한 최종 청구 상한으로 작동하는지도 시험해야 합니다.

현재 확인: 로컬 플랫폼 기동과 일부 기능 검사·관제 첫 화면 캡처. **남은 확인:** 중앙 중계, 모델 호환, 동시 사용, 학생 완주, 비용. 이 문서는 배포 명령 실행 결과나 학생 설치 가이드가 아닙니다.

기술 근거: [Cloud Run 비밀번호 연결](#) · [요청 제한시간](#) · [LiteLLM 가상키](#).
프로젝트 근거: 학교회차 대응표, compose.local.yml, 첫날~다섯째날 기능 기록, 현재 운영 합의. 과거 리뉴얼의 시나리오·가격표는 현재 구현 근거로 사용하지 않음.